

1 Turing Machine

1.1 The basic case of Turing Machine

- The components of TM
 1. An infinite tape
 2. A finite-state control unit(state set)
 3. A single head which reads symbols from the tape and changes symbols on the tape.
- The property of tape
 1. tape 有一个左端点，右端可以无限延伸。为了防止将指针头移除左端，tape 的最左端点加了一个 $\triangleright$ ，一旦指针头读到 $\triangleright$ ，我们就立马将指针左移动一位。
 2. 我们使用 L 和 R 表示指针头向左移动或者向右移动
 3. 纸带上空用 $\sqcup$ 表示。
- The definition of TM

A **Turing Machine**, or **TM**, is a 5-tuple $(Q, \Sigma, \delta, s, H)$ where,

- ① Q is the finite set of states;
- ② Σ is an alphabet that contains $\sqcup$ and $\triangleright$ but not containing the symbols **L** and **R**;
- ③ $s \in Q$ is the initial state;
- ④ H is the set of halting states;
- ⑤ δ is the transition function
 $\delta : (Q \setminus H) \times \Sigma \rightarrow Q \times (\Sigma \cup \{\mathbf{L}, \mathbf{R}\})$ such that:
 - ⑥ If $\delta(q, \triangleright) = (p, b)$ then $b = \mathbf{R}$,
 - ⑦ If $a \in \Sigma$ and $\delta(q, a) = (p, b)$ then $b \neq \triangleright$.

图 1: Component-of-TM

1. Q 是有限状态集合
2. Σ 是包含 $\sqcup$ 和 $\triangleright$ 但是不包含 $\mathbf{L}$ 和 $\mathbf{R}$ 的字母表
3. $s \in Q$ 是初始状态
4. H 是终止状态集合
5. δ 是转移方程，左边是非停机状态 * 字母表的输入组合，右边是新状态 *(要写符号以及移动方向) 的组合

以下是七元组的图灵机：

DEFINITION 3.3

A **Turing machine** is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the **blank symbol** $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{\mathbf{L}, \mathbf{R}\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

图 2: Tuple7ML

1.2 Configuration

- 为了清晰地看清当前图灵机的 Configuration，我们将纸带分为两个部分，左边部分包括从指针头指到的扫描格到最左边，右边部分是不包括当前扫描到的鸽子的扫描格右边的部分。
- Configuration of TM

Configuration of TM

A configuration of a Turing machine $M = (Q, \Sigma, \delta, s, H)$ is a member of $Q \times \triangleright \Sigma^* \times (\Sigma^*(\Sigma \setminus \{\sqcup\}) \cup \{\varepsilon\})$

图 3: Configuration-of-ML

TM 的配置，第一个是状态，第二个是以 $\triangleright$ 开头的由字母表中的符号组合成的字符串，第三个部分前面就是扫描到的表格的右边部分的字符串集合 + 不能以 $\sqcup$ 结尾，结尾要以 ϵ 结尾。

$(q, \triangleright a, aabbb)$ and $(q, \triangleright \sqcup \sqcup b, \epsilon)$ are configurations.

图 4: Configuration-eg

- We could simplify the configuration (q, wa, v) by $(q, w\underline{a}v)$.
- Where underlined position indicates the head position.

图 5: Simplify-Configuration

1.3 Computation

Computation

Computation of TM

For the Turing machine M , let $\vdash_M^*$ be the transitive closure of $\vdash_M$. We say that configuration C **yield** configuration C' if $C \vdash_M^* C'$. A **computation** by M is a sequence of configurations $C_0, \dots, C_n$ such that;

$$C_0 \vdash_M C_1 \vdash_M \dots \vdash_M C_n$$

Example

- Let M be our first example of TM.
- If M started in configuration $(q_1, \triangleright \sqcup \sqcup aa)$ then;

$$(q_1, \triangleright \sqcup \sqcup aa) \vdash_M (q_0, \triangleright \sqcup \underline{a}a)$$

$$\vdash_M (q_1, \triangleright \sqcup \sqcup \underline{a})$$

$$\vdash_M (q_0, \triangleright \sqcup \sqcup \underline{a})$$

$$\vdash_M (q_1, \triangleright \sqcup \sqcup \underline{\sqcup})$$

$$\vdash_M (q_0, \triangleright \sqcup \sqcup \sqcup \underline{\sqcup})$$

$$\vdash_M (q_2, \triangleright \sqcup \sqcup \sqcup \underline{\sqcup})$$

$$\delta(q_0, a) = (q_1, \sqcup)$$

$$\delta(q_0, \sqcup) = (q_2, \sqcup)$$

$$\delta(q_0, \triangleright) = (q_0, \mathbf{R})$$

$$\delta(q_1, a) = (q_0, a)$$

$$\delta(q_1, \sqcup) = (q_0, \mathbf{R})$$

$$\delta(q_1, \triangleright) = (q_1, \mathbf{R})$$

图 6: Computation

1.4 Symbol Writing and Head-Moving

- eg1

对于每一个 a 属于字符集并上 L, R 但是不包括 $\triangleright$. 我们定义一个图灵机 M_a , 状态集合包含 $start$ 和 $halt$ 两个状态, 字符表, 状态转移方程, 开始状态, 结束状态。对于每一个 b 属于字母集但是不包含 $\triangleright$, 状态转移方程如果状态是 s , 当前字母是 b 中的一个字母, 那么变为

halt 状态, 当前字母修改为 a 中的某一个字母。如果当前状态是 s , 当前字母是 $\triangleright$, 那么图灵机自动从开始状态向右移动一位。所以如果 a 属于字母集合中, 那么图灵机就会写下一个 a , 并且叫此图灵机为符号书写机, 如果 a 是 L 和 R 字母中的一个, 我们就向左或者向右移动, 并且称之为 head-moving machine。

1.5 Combining Machines

- 如果 M_1 和 M_2 是图灵机, 那么我们可以将两个图灵机组成一个新的图灵机。
- 从 M_1 到 M_2 , 只有当 M_1 停止之后, M_2 从 M_1 停止的状态和 head 的位置作为初始状态和 head 的位置。
- eg2

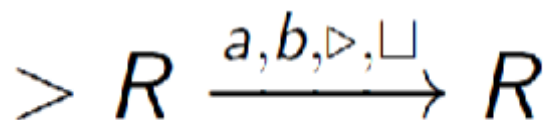


图 7: SimpTMeg1

这张图表示首先将 head 先向右移动一个 square, 如果此 square 是 a , b , 右箭头和 blank, 则再向右移动一个箭头。如果箭头上的字母是字母表中的所有字母, 则可以将字母省略, 只写一个箭头。再进行简化, 将箭头去掉 RR 。最后简化为 R^2 , means that 先将 head 向右移动 2 次。

1.6 Simple Turing Machines

- eg1

以下 TM 表示图灵机将会一直向右进行扫描, 直到遇到 blank, 可以用 $R_{\square}$ 表示。

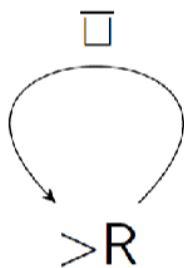


图 8: SimpTM1

- eg2

以下 TM 表示图灵机将会一直向左进行扫描，直到遇到 blank，可以用 $L_{\square}$ 表示。

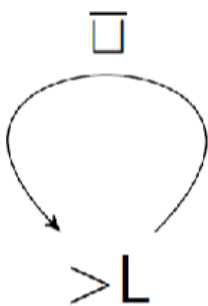


图 9: SimpTM2

- eg3

以下 TM 表示图灵机将会一直向右进行扫描，直到遇到 non-blank，可以用 $R_{\square}$ 表示。

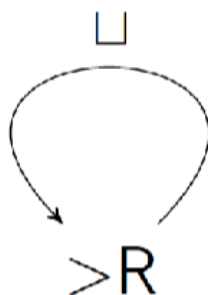


图 10: Simptm3

- eg4

以下 TM 表示图灵机将会一直向左进行扫描, 直到遇到 non-blank, 可以用 $L_{\square}$ 表示。

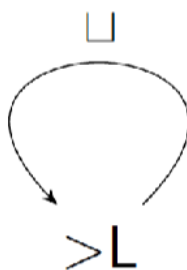


图 11: Simptm4

1.7 Computing With Turing Machine(recognize language)

- 如果一个五元组的图灵机器 $M=(Q, \Sigma, \delta, s, H), H=\{y, n\}$ 。停机状态中的 y 被称为 accept configuration, 停机状态中的 n 被称为 rejecting configuration。
- 让字符集 $\Sigma_0 = \Sigma \setminus \{\square, \triangleright\}$, 图灵机 M 能够 decides 一个语言 $L \subseteq \Sigma_0^*$ 如果对于任意字符串 Σ_0^* 我们有: 如果输入 $w \in L$, 那么 M 停机输出 accept; 如果 $w \notin L$, M 停机输出 reject。
- Recursive Language(递归语言)

1. 一个语言是递归语言，如果有一个图灵机能够判定这个语言。
2. 称一个语言是图灵可判定的 (turing-decidable)，如果某些图灵机可以判定此语言。
3. 如果 L 是一个 recursive language 那么 L 的补也是可递归的。

- Function by TM

定义: M 是五元组图灵机, 字符集 $\Sigma_0 = \Sigma \setminus \{\sqcup, \triangleright\}$ 是输入, 并且 w 是字符集中组成的某个字符串。假设 M 在输入 w 停机, 并且 $(s, \triangleright \sqcup w) \vdash_M^* (h, \triangleright \sqcup y)$ 并且 $y \in \Sigma_0^*$, 那么 y 被称为 M 在输入 w 都输出 (记作 $M(w)$)。

- Recursive Function

定义: 让 f 是从除了空和右三角号的字符集组成的某个字符串到某个字符串的映射。称 M 计算 f , 对于所有 w 属于字符串的子集, $M(w) = f(w)$ 。并且 $f(w)$ 能够被定义仅当 M 在 w 上停机。我们称函数 f 是 recursive。

- Multivariable Functions 计算多元自然数函数

定义: M 是一台五元组图灵机, 字符集包括 $0, 1, ;$ 。并且让函数 f 从 k 个自然数作为输入映射为单个自然数作为输出。我们称 M 计算函数 f 如果对于所有的从 w_1 到 w_k 属于至少含有一个 0 或者 1 的输入, $\text{num}(M(w_1; \dots; w_k)) = f(\text{num}(w_1), \dots, \text{num}(w_k))$ 。我们称此方程 f 是递归的。

- Recursively Enumerable 递归可枚举

定义: M 是五元组的图灵机, 并且 L 是含于不包括空格和空的字符集组成的字符串子集中的一个语言。我们称 M 是 semidecides L 如果对于任何字符串 w 属于字符串子集我们有: w 属于 L 当且仅当 M 在 w 上停机。一个语言是 recursively enumerable/turing recognizable 当且仅当有一个图灵机能够 semidecide 这个语言。

1. 如果 w 属于不包括空格和空的字符集组成的字符串子集, 但是不属于 L , 并且 L 是递归可枚举, 那么 w 将不会到达停机状态。
2. So M semidecides L if for any string $w \in \Sigma_0^*$ we have; $M(w) = \nearrow$ if and only if $w \in L$

- recursive vs recursively enumerable

1. 定理：每一个递归语言都是递归可枚举的。

证明：假设 M 是一个五元组图灵机并且停机状态包含 $n(\text{reject})$ 。有一个 M' 是一个停机状态不包含 n 的五元组并且转换方程中的 (n,a) 变为 (q,a) ，那么状态 n 在 M' 中不是一个停机状态，而是一个循环。所以如果 M 能够判定 L ，那么 M' 能够半判定 L 。所以判定含于半判定。

1.8 Extensions of the turing machine

1.8.1 Multiple Tapes

定义：一个 k -tape 五元组图灵机 M ， k 大于等于 1。转移方程输入部分为当前状态但是不能为停机状态，符号集为当前 k 条扫描符号的 k 元组映射到输出为下一状态， k 条带到动作指令 (包括字符集以及 L , R) 组成的 k 元组

– Configuration of Multiple Tape TM

定义：Let $M = (Q, \Sigma, \delta, s, H)$ be a k -tape Turing machine. A configuration of M is a member of the following set:

$$Q \times (\triangleright \sum^* \times (\sum^* (\sum \setminus \{\sqcup\}) \cup \{\varepsilon\}))^k$$

图灵机当前所处状态 + 第一部分从读写头开始到扫描符号的左侧 (三角号 + 字符串集) + 第二部分从读写头当前扫描符号开始到带右侧的所有符号，若带非空结尾不能为空白符，若带为空用空串符号表示。

By $\vdash_M^*$ we mean the transitive closure (转移关系闭包) of $\vdash_M$. $\vdash_M$ 表示一个 configuration 的转移， $\vdash_M^*$ 表示零次/一次/多次 configuration 的转移。

- 对于多带图灵机，每个动作指令的右上标表示让图灵机的第几个纸带做此操作。
- 每一个多带图灵机都于一个单带图灵机等价。

证明：单带图灵机 M 必须以某种方式包含多带图灵机 M' 当前状态下的所有信息。我们将单带图灵机的纸带划分为 $2k$ 条轨道，奇数条轨道表示从 1 到 k 条纸带，偶数条轨道记录当前指针指在多带每一条带的哪一个位置，就在那个位置下赋值为 1，其余赋值为 0

1.8.2 Two-way Infinite Tape

- 在基础图灵机的定义下，纸带是双向无限长的，并且没有右箭头，纸带从输入的左端开始。

- 定理：每一个双向无限纸带图灵机都可以等价于单纸带图灵机。

证明：使用双带图灵机模拟双向无限纸带图灵机，双带图灵机一条纸带表示从右到输入的纸带，第二条纸带以反向顺序存储指针头左侧的部分。

1.8.3 Two-dimensional Tape

每一步计算都有 4 种可能的操作 $\text{Left, Right, Up, Down}\{L, R, U, D\}$ 。

每一个双维无限纸带图灵机都等价于一台单带图灵机。

1.8.4 All Models Are Equivalent

- Every Nondeterministic Turing machine is equivalent with a deterministic Turing machine.
- Transition function of a Turing machine could be nondeterministic transition, i.e.,

$$\delta : (Q \setminus H) \times \Sigma \rightarrow \mathcal{P}(Q \times (\Sigma \cup \{L, R\}))$$

, P 是幂集。or equivalently a subset of:

$$((Q \setminus H) \times \Sigma) \times (Q \times (\Sigma \cup \{L, R\}))$$

1.9 Nondeterministic Turing Machine 不确定性图灵机

定义：非确定性图灵机是一个五元组 $(K, \Sigma, \Delta, s, H)$, K, Σ, s, H 和标准图灵机相同, Δ 是集合 $((K - H) \times \Sigma) \times (K \times (\Sigma \cup \{L, R\}))$ 的一个子集。

1. 定义: M 是一个五元组 $M = (K, \Sigma, \Delta, s, \{y, n\})$ 的非确定性图灵机。 M 能够判定一个语言 $L \subseteq (\Sigma - \triangleright, \sqcup)^*$ 若以下条件满足 $w \in (\Sigma - \triangleright, \sqcup)^*$:
 - (1) 存在一个自然数 N (人为设定的最大执行步数), 对于给定的图灵机 M 和输入 w , 让机器从初始格局出发, 连续执行 N 步转移后, 无法到达任何合法格局。
 - (2) 当且仅当 $(s, \triangleright \sqcup w) \vdash_M^* (y, u \sqcup v)$, 并且 $u \sqcup v \in \Sigma^*$, $a \in \Sigma$, 此时 $w \in L$ 。我们可以通过设置执行次数上界来让图灵机停机。
2. Definition 4.5.1: Let $M = (K, \Sigma, \Delta, s, H)$ be a nondeterministic Turing machine. We say that M accepts an input $w \in (\Sigma - \triangleright, \sqcup)^*$ if $(s, \triangleright \sqcup w) \vdash_M^* (h, u \sqcup v)$ for some $h \in H$ and $a \in \Sigma$, $u, v \in \Sigma^*$. Notice that a nondeterministic machine accepts an input even though it may have many nonhalting computations on this input We say that M semidecides a language L if the following holds for all $w \in (\Sigma - \triangleright, \sqcup)^*$: $w \in L$ if and only if M accepts w
3. 要使一台非确定型图灵机完成语言判定与函数计算, 我们要求其所有计算过程均需停机。这一点可通过如下假设实现: 不存在任何计算过程的执行步数超过 N ——其中 N 是一个上界, 其取值由图灵机本身与输入串共同决定。其次, 若要让图灵机 M 实现语言判定, 我们仅需满足一个条件即可: 在其所有可能的计算路径中, 至少存在一条路径最终接受该输入串。至于其余的计算路径, 无论部分、大部分乃至全部路径最终判定为拒绝, 均不影响语言判定的结果。
4. M computes 一个方程 $f : (\Sigma - \{\triangleright, \sqcup\})^* \mapsto (\Sigma - \{\triangleright, \sqcup\})^*$ 若对于所有的 $w \in (\Sigma - \triangleright, \sqcup)^*$ 以下两个条件均满足
 - (a) (a) There is an N , depending on M and w , such that there is no configuration C satisfying $(s, \triangleright \sqsubseteq w) \vdash_M^N C$
 - (b) (b) $(s, \triangleright \sqcup w) \vdash_M^* (h, u \sqcup v)$ if and only if $ua = \triangleright \sqcup$, and $v = f(w)$ 。
停机时, 读写头左侧符号串 + 扫描符号恰好等于左端符号 $\triangleright$ 加空

白符 $\sqcup$ ，读写头右侧的符号串恰好等于函数 f 在输入 w 上的输出值。

5. 定理：如果一个非确定性图灵机 M 半判定或者判定一个语言或者能够计算一个函数，那么会有一个标准的图灵机 M' 判定或者半判定相同的语言或者计算相同的函数。

证明： M' 将会尝试所有 M 所做的可能的计算，如果发现了 M 的停机配置， M' 同样也会停机

6. 定理：每一个非确定性图灵机都有一个等价的确定性图灵机。

证明：非确定型图灵机 N 在输入串 w 上的计算过程视为一棵计算树。树的每一条分支对应非确定性的一个计算分支，每一个节点对应 N 的一个格局，树根则是初始格局。确定性图灵机 D 会在这棵树中搜索接受格局。如果使用深度优先搜索策略，该策略会先沿着一条分支探索至尽头，再回溯探索其他分支。若 D 采用这种方式，可能会陷入某条无限长的分支中无限循环，从而错过其他分支上的接受格局。因此，我们选择广度优先搜索——这种策略会逐层探索所有分支，保证在遇到接受格局之前， D 能访问到树中的每一个节点。

7. 定理：如果非确定性图灵机对于某一类语言的所有输入在所有路径上都能停机，则被称为 decider。
8. 推论：一个语言是图灵可识别的当且仅当如果某个不确定性图灵机能够识别它。
9. 推论：一个语言是可判定的当且仅当非确定性图灵机可以判定它。

1.10 Recursive Function 递归方程

- Basic Function

1. K -ary zero function(k 元零函数)

$$zero_k(n_1, \dots, n_k) = 0, \quad n_1, \dots, n_k \in \mathbb{N}$$

2. K -ary projection function(k 元映射函数)

$$p_j(n_1, \dots, n_k) = n_j \quad n_1, \dots, n_k \in \mathbb{N}$$

3. successor function(后继函数)

$$succ(n) = n + 1, n \in \mathbb{N}$$

4. Primitive recursive(原始递归函数)

定义: Basic function 是 primitive recursive

- (a) If $g : \mathbb{N}^k \rightarrow \mathbb{N}$ and $h_1 : \mathbb{N}^s \rightarrow \mathbb{N}, \dots, h_k : \mathbb{N}^s \rightarrow \mathbb{N}$ are primitive functions then following function is also primitive recursive:

$$f(n_1, \dots, n_s) = g(h_1(n_1, \dots, n_s), \dots, h_k(n_1, \dots, n_s))$$

- (b) If $g : \mathbb{N}^k \rightarrow \mathbb{N}$ and $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ are primitive recursive, then the function defined recursively by g and h is also primitive recursive:

$$f(n_1, \dots, n_k, 0) = g(n_1, \dots, n_k)$$

$$f(n_1, \dots, n_k, m+1) = h(n_1, \dots, n_k, m, f(n_1, \dots, n_k, m))$$

第一个变量描述的是第一个输入, 第二个变量 m 感觉像是描述递归的次数, 第三个变量描述前一次递归求出来的数值。

- Primitive recursive predicate(原始递归谓词)

任何方程 f 从 $\mathbb{N}$ 元输入, 输出 0 或者 1, 我们称这个方程为原始递归谓词

- Function Defined by Case(分段函数)

我们通过 primitive recursive predicate 可以构造出分段函数, 如果多元函数输出为 1, 则执行第一个函数, 若多元输出为 0, 则执行另一个函数。这个分段函数也可以只用一个函数进行表示。

- Minimalization 极小化函数

函数 g 是从 $k+1$ 元输入映射到一元输入。函数 f 从 k 元输入映射到一元输入。 g 的 $k+1$ 元输入是固定值, 从 $m=0$ 开始试 m 的值, 第一个让 $g(n_1, \dots, n_k, m) = 1$ 的 m 就是 f 的值。

- μ -recursive

定义: 如果一个方程的极小化算法总能终止, 那么这个函数被称为可极小化的。一个方程是 μ -recursive, 如果一个函数对某一可极小化函数执行极小化操作之后得到原始递归函数。

1. 定理: 如果一个函数 f 从 k 元映射到一元可以被图灵机可计算, 那么这个函数是 μ -recursive, 并且是 recursive。

- If g and $h_1, \dots, h_s$ are computable by a Turing machine then

$$f(n_1, \dots, n_k) = g(h_1(n_1, \dots, n_k), \dots, h_s(n_1, \dots, n_k))$$

computable by a Turing machine.

- If f is defined recursively from Turing computable functions g and h , i.e.,

$$f(n_1, \dots, n_k, 0) = g(n_1, \dots, n_k)$$

$$f(n_1, \dots, n_k, \underline{m+1}) = h(n_1, \dots, n_k, \underline{m}, f(n_1, \dots, n_k, \underline{m}))$$

1.11 Grammer

定义: 一个文法/无约束文法 (unrestricted grammer)/重写系统 (rewrite system) 是一个 quadruple 四元组 $G = (V, \Sigma, R, S)$, V 是字母集, $\Sigma \subseteq V$ 是 terminal symbols 终结字符集, $V - \Sigma$, is the set of nonterminal symbols. $S \in V - \Sigma$ is start symbol. R 是 set of rules. R , the set of rules, is a finite subset of $(V^*(V - \Sigma)V^*) \times V^*$

1.12 Random Access